

Week 1 Structured Notes: Basics of Arrays

Topic Overview

Week 1 introduces the basic idea of arrays in C programming. Before learning advanced topics like memory mapping, stack arrays, heap arrays, resizing, and compiler formulas, you must first understand what an array is, how it is created, how values are stored, and how values are accessed using indexes.

This week mainly covers:

1. Variables and arrays
 2. Scalar and vector variables
 3. Array declaration
 4. Array initialization
 5. Indexing
 6. Traversing arrays
 7. Finding sum, maximum, and minimum values
 8. Printing memory addresses
 9. Contiguous memory
 10. Basic time complexity: $O(1)$ and $O(N)$
-

1. Variables

Meaning of a Variable

A variable is a named memory location that stores one value.

Example:

```
int x = 10;
```

Here, `x` is the variable name, and it stores the value `10`.

You can imagine it like a small box:

```
x  
[10]
```

Only one value is stored inside this box.

Scalar Variable

A variable that stores only one value is called a scalar variable.

Example:

```
int age = 20;
```

Here, `age` stores only one number.

2. Arrays

Meaning of an Array

An array is a collection of values stored under one name.

Example:

```
int A[5] = {2, 4, 6, 8, 10};
```

Here, `A` is the array name. It stores five integer values.

```
A[0] = 2  
A[1] = 4  
A[2] = 6  
A[3] = 8  
A[4] = 10
```

Simple View of an Array

Index:	0	1	2	3	4
Value:	2	4	6	8	10

The position number is called the index.

Important: In C, array indexing starts from 0.

So:

A[0] means the first element.
A[1] means the second element.
A[2] means the third element.

For an array with 5 elements, the valid indexes are:

0, 1, 2, 3, 4

The index `5` is not valid for a 5-element array.

3. Why Arrays Are Useful

Without arrays, storing many values becomes messy.

Example without an array:

```
int mark1 = 80;  
int mark2 = 75;  
int mark3 = 90;  
int mark4 = 65;  
int mark5 = 88;
```

This works, but it is difficult to manage.

Using an array:

```
int marks[5] = {80, 75, 90, 65, 88};
```

Now all marks are stored under one name: `marks`.

This makes the program cleaner and easier to process using loops.

4. Difference Between Variable and Array

Feature	Variable	Array
Stores	One value	Many values

Feature	Variable	Array
Example	<code>int x = 10;</code>	<code>int A[5] = {1,2,3,4,5};</code>
Access method	By variable name	By array name and index
Example access	<code>x</code>	<code>A[0]</code> , <code>A[1]</code>

Simple explanation:

Variable = one box
 Array = many boxes under one name

5. Array Declaration

Declaration means creating an array.

Example:

```
int A[5];
```

This creates an integer array with 5 spaces.

The array has:

```
A[0]
A[1]
A[2]
A[3]
A[4]
```

But the values are not safely set yet.

Important Warning

If you declare an array without giving values, it may contain garbage values.

Example:

```
int A[5];
```

Possible values may look like this:

```
A[0] = 32764  
A[1] = -10  
A[2] = 0  
A[3] = 999  
A[4] = 45
```

These are not values you entered. They are old values already present in memory.

So, do not assume an uninitialized array contains zero.

6. Array Initialization

Initialization means giving starting values to an array.

There are five common ways to create and initialize arrays.

Method 1: Declaration Only

```
int A[5];
```

This creates 5 integer spaces.

But values may be garbage.

Use this only when you plan to assign values later.

Method 2: Full Initialization

```
int B[5] = {1, 2, 3, 4, 5};
```

All 5 values are given.

```
B[0] = 1  
B[1] = 2  
B[2] = 3  
B[3] = 4  
B[4] = 5
```

This is safe because every element has a known value.

Method 3: Partial Initialization

```
int C[10] = {2, 4, 6};
```

Only the first three values are given.

The remaining values automatically become zero.

```
C[0] = 2  
C[1] = 4  
C[2] = 6  
C[3] = 0  
C[4] = 0  
C[5] = 0  
C[6] = 0  
C[7] = 0  
C[8] = 0  
C[9] = 0
```

Method 4: Zero Initialization

```
int D[5] = {0};
```

This sets all elements to zero.

```
D[0] = 0  
D[1] = 0  
D[2] = 0
```

```
D[3] = 0  
D[4] = 0
```

This is useful when you want a clean array before processing.

Method 5: Auto-Sized Array

```
int E[] = {1, 2, 3, 4, 5, 6};
```

Here, you do not give the size.

The compiler counts the values automatically.

Since there are 6 values, the size of array `E` becomes 6.

7. Accessing Array Elements

Accessing means getting one value from the array.

Example:

```
int A[5] = {2, 4, 6, 8, 10};  
  
printf("%d", A[2]);
```

Output:

```
6
```

Why?

```
A[0] = 2  
A[1] = 4  
A[2] = 6
```

So, `A[2]` gives the third value.

8. Common Indexing Mistake

For this array:

```
int A[5] = {2, 4, 6, 8, 10};
```

The valid indexes are:

```
A[0], A[1], A[2], A[3], A[4]
```

Wrong:

```
A[5]
```

`A[5]` is outside the array.

This can cause incorrect output or program errors.

9. Traversing an Array

Traversal means visiting every element in an array one by one.

A loop is normally used for traversal.

Example:

```
int A[5] = {2, 4, 6, 8, 10};

for (int i = 0; i < 5; i++) {
    printf("%d ", A[i]);
}
```

Output:

```
2 4 6 8 10
```


How the Loop Works

```
i = 0 -> A[0] = 2
i = 1 -> A[1] = 4
i = 2 -> A[2] = 6
i = 3 -> A[3] = 8
i = 4 -> A[4] = 10
```

When `i` becomes 5, the condition `i < 5` becomes false, so the loop stops.

10. Finding the Sum of Array Elements

To find the sum, create a variable called `sum` and add each array element into it.

Example:

```
#include <stdio.h>

int main() {
    int A[5] = {2, 4, 6, 8, 10};
    int sum = 0;

    for (int i = 0; i < 5; i++) {
        sum = sum + A[i];
    }

    printf("Sum = %d", sum);

    return 0;
}
```

Output:

```
Sum = 30
```

Step-by-Step Calculation

```
sum = 0
sum = 0 + 2 = 2
sum = 2 + 4 = 6
```

```
sum = 6 + 6 = 12  
sum = 12 + 8 = 20  
sum = 20 + 10 = 30
```

Final sum is 30.

11. Finding the Maximum Value

Maximum means the biggest value in the array.

Example:

```
#include <stdio.h>  
  
int main() {  
    int A[5] = {2, 4, 6, 8, 10};  
    int max = A[0];  
  
    for (int i = 1; i < 5; i++) {  
        if (A[i] > max) {  
            max = A[i];  
        }  
    }  
  
    printf("Maximum = %d", max);  
  
    return 0;  
}
```

Output:

```
Maximum = 10
```

Logic

First, assume the first value is the maximum:

```
int max = A[0];
```

Then compare the other values one by one.

If a bigger value is found, update `max`.

12. Finding the Minimum Value

Minimum means the smallest value in the array.

Example:

```
#include <stdio.h>

int main() {
    int A[5] = {2, 4, 6, 8, 10};
    int min = A[0];

    for (int i = 1; i < 5; i++) {
        if (A[i] < min) {
            min = A[i];
        }
    }

    printf("Minimum = %d", min);

    return 0;
}
```

Output:

```
Minimum = 2
```

Logic

First, assume the first value is the minimum:

```
int min = A[0];
```

Then compare the other values one by one.

If a smaller value is found, update `min`.

13. Memory Addresses of Array Elements

Every value in an array is stored somewhere in memory.

You can print the address of each element using `&`.

Example:

```
#include <stdio.h>

int main() {
    int A[5] = {2, 4, 6, 8, 10};

    for (int i = 0; i < 5; i++) {
        printf("A[%d] = %d, Address = %p\n", i, A[i], (void*)&A[i]);
    }

    return 0;
}
```

Possible output:

```
A[0] = 2, Address = 0061FF00
A[1] = 4, Address = 0061FF04
A[2] = 6, Address = 0061FF08
A[3] = 8, Address = 0061FF0C
A[4] = 10, Address = 0061FF10
```

Your computer may show different addresses. That is normal.

The important thing is the pattern.

14. Contiguous Memory

Contiguous memory means memory locations are placed side by side.

In an integer array, if one integer takes 4 bytes, the address usually increases by 4 for each next element.

Example:

```
A[0] address = 200  
A[1] address = 204  
A[2] address = 208  
A[3] address = 212  
A[4] address = 216
```

Each address increases by 4 because each integer uses 4 bytes.

This is why arrays can access elements quickly.

15. Access Time: $O(1)$

Accessing one array element is fast.

Example:

```
A[3]
```

The computer can directly access `A[3]` using its index.

It does not need to check all earlier elements first.

So, accessing one element is called:

```
O(1)
```

This means constant time.

16. Traversal Time: $O(N)$

Traversal means visiting every element.

Example:

```
for (int i = 0; i < 5; i++) {  
    printf("%d", A[i]);  
}
```

If the array has 5 elements, the loop runs 5 times.

If the array has 100 elements, the loop runs 100 times.

So, traversal is:

$O(N)$

This means the time depends on the number of elements.

17. Full Week 1 Practice Program

This program covers printing values, printing addresses, finding sum, maximum, and minimum.

```
#include <stdio.h>

int main() {
    int A[5] = {2, 4, 6, 8, 10};

    int sum = 0;
    int max = A[0];
    int min = A[0];

    printf("Array values:\n");

    for (int i = 0; i < 5; i++) {
        printf("A[%d] = %d, Address = %p\n", i, A[i], (void*)&A[i]);

        sum = sum + A[i];

        if (A[i] > max) {
            max = A[i];
        }

        if (A[i] < min) {
            min = A[i];
        }
    }

    printf("\nSum = %d\n", sum);
    printf("Maximum = %d\n", max);
    printf("Minimum = %d\n", min);
}
```

```
    return 0;  
}
```

18. Expected Output

The address values can change from computer to computer.

Example:

```
Array values:  
A[0] = 2, Address = 0061FF00  
A[1] = 4, Address = 0061FF04  
A[2] = 6, Address = 0061FF08  
A[3] = 8, Address = 0061FF0C  
A[4] = 10, Address = 0061FF10  
  
Sum = 30  
Maximum = 10  
Minimum = 2
```

19. Key Terms

Term	Simple Meaning
Variable	Stores one value
Array	Stores many values under one name
Index	Position number of an array element
Element	One value inside an array
Traversal	Visiting every array element one by one
Initialization	Giving starting values to an array
Garbage value	Unknown value already present in memory
Contiguous memory	Memory stored side by side
O(1)	Constant-time access
O(N)	Time depends on number of elements

20. Week 1 Summary

By the end of Week 1, you should understand that:

1. A variable stores one value.
 2. An array stores many values under one name.
 3. Array indexes start from 0 in C.
 4. A 5-element array has indexes 0 to 4.
 5. Arrays should be initialized before use.
 6. Uninitialized arrays may contain garbage values.
 7. A loop is used to traverse an array.
 8. Sum, maximum, and minimum can be found by looping through the array.
 9. Array elements are stored in contiguous memory.
 10. Accessing one array element is $O(1)$.
 11. Traversing the whole array is $O(N)$.
-

21. Week 1 Checkpoint Questions

Try answering these without looking at the notes.

1. What is a variable?
 2. What is an array?
 3. What is the difference between a variable and an array?
 4. Why does array indexing start from 0?
 5. What are the valid indexes of `int A[5]`?
 6. What is an uninitialized array?
 7. What is a garbage value?
 8. What is full initialization?
 9. What is partial initialization?
 10. What is zero initialization?
 11. What is an auto-sized array?
 12. What does traversal mean?
 13. How do you find the sum of array elements?
 14. How do you find the maximum value?
 15. How do you find the minimum value?
 16. What does contiguous memory mean?
 17. Why does the address increase by 4 in an integer array?
 18. What does $O(1)$ mean?
 19. What does $O(N)$ mean?
 20. Why are loops important when using arrays?
-

22. Practice Task

Write a C program that does the following:

1. Create an array of 5 marks.
2. Print all marks.
3. Find the total marks.
4. Find the average mark.
5. Find the highest mark.
6. Find the lowest mark.
7. Print the memory address of each element.

This task covers almost all concepts from Week 1.

23. Simple Final Memory Aid

```
Variable = one value  
Array = many values  
Index = position  
A[0] = first element  
Traversal = visit all elements  
Sum = add all values  
Max = biggest value  
Min = smallest value  
Address = memory location  
Contiguous = side by side  
O(1) = direct access  
O(N) = loop through all elements
```